

STARSAI — Software Development Life Cycle

Project: STARS — Safety Transit Analytics & Risk System **Product:** Toronto Nighttime Transit Safety Index (T-NTSI) **Team:** STARS AI (Jasdish Singh, Centennial College + Tiana Gaurd, UofT) **Competition:** Transit Data Challenge 2026 **Submission Window:** May 15–30, 2026 **Document version:** 1.0 — 2026-05-14 **Repository:** <https://github.com/Jasdish-Singh/STARSAI>

1. Project Overview

1.1 Vision

A defensible, auditable, open-data composite index of nighttime transit-stop safety for women in Toronto — published as a public policy instrument and an interactive web tool — that demonstrates how routine open data can be recombined into action-grade civic intelligence without invading anyone’s privacy.

1.2 Mission

Score every TTC stop (~12,000 surface stops + 75 subway/RT stations) on a 0–100 nighttime safety index across 24 hourly bins and 3 day-types, expose the score with full factor-level provenance, and let any user or policymaker simulate interventions (“what if we lit Dufferin?”) in the browser.

1.3 Problem Statement

Women restrict, re-route, or refuse nighttime transit at materially higher rates than men. Toronto publishes the data to quantify *why* — crime, lighting, isolation, frontage, demographics — but no one has fused it into a single, transparent, time-aware index of stop-level risk. Existing safety apps are either crowdsourced and biased (Citizen, SketchFactor) or proprietary and opaque (in-house transit-agency dashboards). We build the open alternative.

1.4 Success Metrics

Metric	Target	Measurement
Stops scored	12,000	Count of stops in <code>index.geojson</code>
Time bins per stop	24 hours × 3 day-types = 72	Schema check
Factor count	8 of the planned 10	Pipeline output schema
Score auditability	100% — every score deep-links to raw rows	Manual QA on 50 sampled stops
Frontend p95 load	< 2.5 s on 4G	Lighthouse / WebPageTest

Metric	Target	Measurement
Map interactivity	60 fps pan/zoom on 12k stops	DevTools FPS monitor
WCAG conformance	2.1 AA	axe-core CI gate
Equity disclosure	Score-residual-by-income panel in UI	UI feature present
Competition outcome	Top-3 finalist OR top-5 OR submission complete with all deliverables — three tiers	Judge feedback

1.5 Stakeholders

Group	Interest	Engagement
Transit Data Challenge judges	Innovation, rigor, social impact	Final submission + 3-min demo video
TTC / City of Toronto	Operational utility, defensibility	Demoable scenario simulator
Women transit riders	Practical safety insight	Public web app, route mode
Open data community	Reproducibility, licensing	Public GitHub, MIT + ODbL where appropriate
Academic reviewers	Methodology	Technical report + appendix

1.6 Users

- **Primary:** Female-identifying transit riders making post-9 PM journeys.
- **Secondary:** City planners, councillors' staff, TTC service planning, academic researchers, journalists.
- **Tertiary:** Curious technically-literate Torontonians.

2. Requirements

2.1 Functional Requirements

ID	Description	Priority	Acceptance Criteria
FR-001	System shall ingest TTC GTFS static feed	P0	stops.txt parsed; 12,000 stop rows in canonical store
FR-002	System shall ingest Toronto Police MCI dataset	P0	36 months of incident rows; only occurrencehour [21,4) used for night scoring
FR-003	System shall ingest Toronto Street Lighting open data	P0	Point dataset loaded; counts per H3-r10 hex computed
FR-004	System shall ingest OSM POI data filtered to night-relevant tags	P0	Pulled via Overpass; categorized (active frontage, alcohol, transit-adjacent)
FR-005	System shall ingest Toronto 311 service request data	P1	Last 12 months pulled; disorder categories filtered
FR-006	System shall ingest StatCan census-tract income & population	P0	2021 census joined to hex grid for equity layer
FR-007	System shall ingest TTC GTFS-Realtime feed for live wait exposure	P2	Cron job updates every 30s; live overlay endpoint returns < 200ms
FR-008	System shall compute lighting density factor per stop	P0	Float [0,100], log-scaled within 75m buffer, schema validated
FR-009	System shall compute crime history factor per stop	P0	Decay-weighted MCI count within 250m, 9 PM-4 AM, last 36 months

ID	Description	Priority	Acceptance Criteria
FR-010	System shall compute eyes-on-street factor per stop	P0	Count of OSM venues with opening_hours covering query bin, within 150m
FR-011	System shall compute physical isolation factor per stop	P0	Inverse of POI count + setback proxy via street width
FR-012	System shall compute wait-exposure factor per stop per bin	P0	$E[\text{wait}] \times$ isolation; uses GTFS headway if RT unavailable
FR-013	System shall compute sightline factor per stop	P1	OSM building footprint occlusion within 50m forward arc
FR-014	System shall compute pedestrian co-presence forecast	P1	Poisson GAM trained on TTC boardings + POI density; bin-aware
FR-015	System shall compute crossing-danger factor per stop	P1	Distance to nearest signalized crossing + KSI (Killed/Seriously Injured) collision density
FR-016	System shall compute 311 disorder factor per stop	P2	Count of disorder categories in last 90 days within 200m
FR-017	System shall compute alcohol-outlet density factor per stop	P2	AGCO licensed premises within 200m, capped

ID	Description	Priority	Acceptance Criteria
FR-018	System shall compute composite T-NTSI score per (stop, hour, day-type)	P0	Weighted sum, output [0,100], 100 = safest
FR-019	System shall persist full computation provenance per score	P0	<code>provenance.json</code> per stop with factor inputs and source row IDs
FR-020	Frontend shall render all stops on an interactive map	P0	<code>Deck.gl ScatterplotLayer</code> 12k points, smooth pan/zoom
FR-021	Frontend shall provide hour-of-day time slider	P0	Slider 21→04; map recolors within 100ms
FR-022	Frontend shall provide day-type selector (weekday/Fri-Sat/Sun)	P0	Three-way toggle; map updates
FR-023	Frontend shall show factor breakdown panel on stop click	P0	Modal lists all factors with bars and raw values
FR-024	Frontend shall provide a route safety lookup (“from→to at time”)	P1	Returns ordered list of stop/segment scores with summary score
FR-025	Frontend shall expose a Policy Simulator panel	P1	Sliders for lighting/POI; live re-score; equity delta readout
FR-026	Frontend shall expose an Equity Mode toggle	P0	Color stops by score-residual-vs-income; sidebar narrative

ID	Description	Priority	Acceptance Criteria
FR-027	Frontend shall expose Score Provenance deep links to open data	P0	Each factor row links to dataset URL on open.toronto.ca
FR-028	Frontend shall provide LLM Route Auditor narration	P1	Streams natural-language route audit, constrained to index data
FR-029	Frontend shall provide voice query mode	P2	Web Speech API STT + TTS for route queries
FR-030	Frontend shall provide “Last 200m” detail view on stop click	P1	Zoom-to-stop overlay with lights, CCTV, businesses, 311
FR-031	System shall publish a static JSON API of scores	P0	<code>/data/scores.json</code> (gzipped < 5 MB), regenerated nightly
FR-032	System shall publish a methodology page	P0	<code>/methodology</code> route with weights, formulas, validation
FR-033	System shall publish a downloads page	P1	CSV / GeoJSON downloads of full dataset, MIT-licensed
FR-034	System shall support reproducible re-build via <code>make all</code>	P0	Single command rebuilds entire pipeline from raw → published

2.2 Non-Functional Requirements

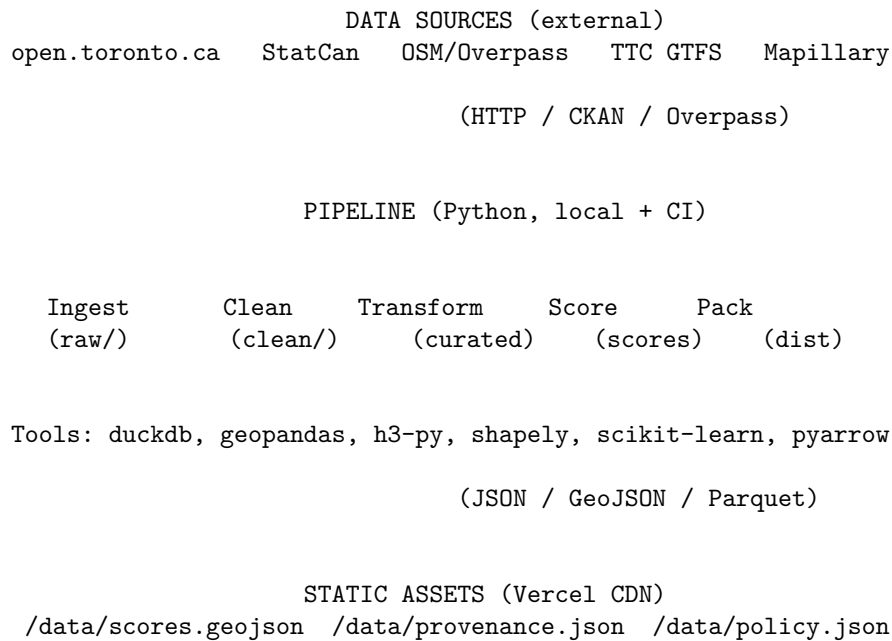
ID	Description	Target	Verification
NFR-001	Page p95 load time on 4G	2.5 s	Lighthouse CI

ID	Description	Target	Verification
NFR-002	Score computation full rebuild	15 min on 8-core laptop	Pipeline timing log
NFR-003	Map interactivity	50 fps median pan/zoom with 12k points	Chrome DevTools
NFR-004	Initial JS bundle	300 KB gzipped	Bundlephobia / Webpack analyzer
NFR-005	Scores GeoJSON payload	5 MB gzipped (full); 1.5 MB (binned MVT alt)	Build artifact size check
NFR-006	WCAG conformance	2.1 AA	axe-core, manual screen-reader smoke test
NFR-007	Keyboard navigation	100% interactive controls reachable	Manual QA checklist
NFR-008	Colorblind safety	Viridis / Cividis palette, no red-green	Sim-Daltonism inspection
NFR-009	Data freshness	Crime 7 days lag; lights/POI 30 days	Pipeline timestamp metadata
NFR-010	Pipeline determinism	Same input → byte-identical output	Hash check on two consecutive runs
NFR-011	Privacy	No user PII ever stored server-side	Code review + privacy section in report
NFR-012	Licensing compliance	All data sources cited with license type	Attributions page
NFR-013	Browser support	Last 2 versions Chrome, Firefox, Safari, Edge	BrowserStack smoke
NFR-014	Mobile usability	Functional on 375×667 viewport	Real-device test on iPhone SE / mid Android
NFR-015	Offline robustness	App shell + last-fetched scores cached	Service worker, Lighthouse PWA score 80

ID	Description	Target	Verification
NFR-016	Code quality	70% test coverage on pipeline & scoring	pytest -cov
NFR-017	Reproducibility	Pinned deps, locked Python/Node versions	uv.lock, pnpm-lock.yaml, .tool-versions
NFR-018	Documentation	README + SDLC + methodology + ADRs	Files exist in /docs

3. System Architecture

3.1 Component Diagram



WEB APP (React + Vite, SSR'd)

Map (deck.gl)	Time Slider	Equity Mode	Policy Sim	LLM Audit
------------------	----------------	----------------	---------------	--------------

State: Zustand Map: Mapbox GL JS Charts: D3

EDGE FUNCTIONS (Vercel - optional, P1+P2 only)
/api/llm-narrate /api/route-score /api/gtfs-rt /api/dp-report

3.2 Data Flow Diagram

```
[raw .csv/.zip/.geojson]
    python -m starsai.ingest

[bronze/ parquet, schema-validated]
    python -m starsai.clean    (drop nulls, normalize CRS to EPSG:4326)

[silver/ parquet, deduped]
    python -m starsai.transform    (H3 r10 binning, buffers, joins)

[gold/ parquet, per-stop feature table]
    python -m starsai.score    (factor compute → composite)

[scores/ scores.parquet + provenance.parquet]
    python -m starsai.pack    (parquet → geojson + topojson)

[dist/ scores.geojson, provenance.json, policy.json]
    vercel deploy --prod

[CDN: vercel.app/data/*]
    fetch() in browser

[React app rendering]
```

3.3 Technology Stack

Layer	Choice	Why
Pipeline language	Python 3.12	Best geospatial ecosystem (geopandas, h3, shapely)
Local data engine	DuckDB 1.x	SQL on parquet, zero-server, fast spatial joins via spatial extension
Geo lib	GeoPandas + Shapely 2	Standard; Shapely 2 is vectorized & fast
Spatial index	H3 r10 (~65m edge)	Native to Uber, ideal for fine-grained urban analysis
ML	scikit-learn + pyTorch (only if GNN stretch)	sklearn for GAM/RF; torch only if Idea #6 happens
Frontend framework	React 18 + Vite	Fast HMR, smaller bundles than Next default
Map renderer	Mapbox GL JS + Deck.gl 9	Industry standard; deck.gl handles 12k points natively
State	Zustand	1 KB, no boilerplate vs Redux
Charts	D3 (only what's needed)	Hand-crafted; better than dragging in Plotly
Hosting	Vercel (free tier)	Edge functions + CDN + GitHub integration; beats Render for static + serverless
CI	GitHub Actions	Free, integrated, matrix builds
LLM	Claude Haiku 4 via Anthropic API	Cheap, fast streaming, constrained-tool-use
Auth	None	No user accounts; aligns with NFR-011

3.4 Infrastructure Diagram

GitHub repo (main)

push

GitHub Actions

- lint+test (PR gate)
- nightly data refresh (cron)
- on tag → Vercel prod deploy

Vercel CDN static	Vercel Edge fn /api/*
----------------------	--------------------------

Browser (user)

4. Data Architecture

4.1 Data Sources

ID	Dataset	Source	API / URL	Schema (key fields)	Refresh	Resolution	License
DS-01	TTC GTFS static	open.toronto.ca	https://open.toronto.ca/api/v2/ttc/gtfs/	stop_id, stop_lat, stop_lon, stop_name	Weekly	Point	OGL- Toronto
DS-02	TTC GTFS- Realtime	TTC	https://realtime.toronto.ca/gtfs/	trip_id, stop_id, stop_name	Realtime	Point	OGL- Toronto
DS-03	Toronto Police MCI	open.toronto.ca	https://open.toronto.ca/api/v2/toronto-police/mci/	occurrence-date, hour, lat, long	Quarterly	Point (offset for privacy)	OGL- Toronto
DS-04	Toronto Street Light- ing	open.toronto.ca	https://open.toronto.ca/api/v2/toronto-street-lighting/	lamp_id, Annual lamp_type	Annual	Point	OGL- Toronto

ID	Dataset	Source	API / URL	Schema (key fields)	Refresh	Resolution	License
DS-05	Toronto CCTV cam- eras	open.toronto.ca	based-circuitry-leveraging	location, latitude, longitude	Annual	Point	OGL-Toronto
DS-06	Toronto 311 service re- quests	open.toronto.ca	311-service-requests	lat, long, created_date	Push, custom	Point	OGL-Toronto
DS-07	Toronto KSI colli- sions	open.toronto.ca	motor-vehicle-collisions	hour, geometry	Annual	Point	OGL-Toronto
DS-08	Toronto Neigh- bour- hoods 158	open.toronto.ca	neighbourhoods	name, geometry	Annual	Polygon	OGL-Toronto
DS-09	StatCan Census 2021 — Tract income & pop- ulation	statcan.gc.ca	Profile of CTs table 98-401-X2021006	DAUID, CTUID, median income, pop	5-year	Polygon (tract)	OGC-StatCan
DS-10	OSM POIs	overpass-api.de	Overpass QL to amenity/shop/leisure	amenity, opening_hours, geometry	Continuous	Point + polygon	ODbL
DS-11	AGCO li- censed premises	data.ontario.ca	liquor-sales-licences	status, address	Quarterly	Point (geocoded)	Ontario Open Data
DS-12	Mapillary night- time im- agery	mapillary.com	Mapillary Graph API v4	image_id, geometry, captured_at	Continuous	Point + image	CC-BY-SA
DS-13	Toronto Wards	open.toronto.ca	city-wards	area_name, geometry	Annual	Polygon	OGL-Toronto

4.2 Pipeline Stages

INGEST (starsai/ingest)

download_ckan.py	- CKAN resource fetch, ETag caching to data/raw/
download_overpass.py	- Overpass QL queries to data/raw/osm/
download_statcan.py	- StatCan Web Data Service pulls
manifest.json	- records source URL, fetched_at, sha256 per file

CLEAN (starsai/clean)

normalize_crs.py	- all geometry → EPSG:4326
dedupe.py	- dedupe by source-specific keys
time_normalize.py	- parse to UTC, derive local hour Toronto
validate.py	- pandera schemas, fail loud

TRANSFORM (starsai/transform)

hex_grid.py	- generate Toronto-bbox H3 r10 cells (~22k cells)
buffers.py	- pre-compute 75m / 150m / 250m buffers per stop
temporal_bins.py	- assign (hour, day_type) labels
stop_features.py	- join data into per-stop feature table
pois_classify.py	- apply rule-based POI typology (frontage/alcohol/transit)

SCORE (starsai/score)

factors/	
lighting.py	
crime.py	
eyes_on_street.py	
isolation.py	
wait_exposure.py	
sightline.py	
crossing_danger.py	
disorder_311.py	
alcohol_density.py	
composite.py	- weighted sum with theory + data calibrated weights
weights.yaml	- declarative weight config; checked in
provenance.py	- emit per-stop provenance.json

PACK (starsai/pack)

to_geojson.py	- output dist/scores.geojson
to_topojson.py	- optional compressed alt
to_mvt.py	- vector tiles for very-low-bandwidth
manifest_dist.json	- output catalog with checksums

4.3 H3 Spatial Strategy

- Grid: **H3 resolution 10** (avg edge ~66 m, area ~15,000 m²). Right size for stop-level features.

- Each TTC stop is anchored to its parent r10 hex.
- Neighborhood operations use **k-ring** of size 1 (200 m radius) for “eyes on street” and “co-presence”, **k-ring of 3** (500 m) for crime density.
- Lights/POIs/incidents are counted via spatial buffer (true meters) for the **factor** computation, but the underlying display layer uses H3 hexes for fast frontend rendering of contextual heatmaps.

4.4 Output JSON Schema (served to frontend)

```
// dist/scores.geojson - FeatureCollection
{
  "type": "FeatureCollection",
  "metadata": {
    "schema_version": "1.0",
    "generated_at": "2026-05-22T03:14:00Z",
    "pipeline_git_sha": "ab12cd34",
    "stop_count": 12087
  },
  "features": [
    {
      "type": "Feature",
      "geometry": { "type": "Point", "coordinates": [-79.3849, 43.6532] },
      "properties": {
        "stop_id": "11234",
        "stop_name": "Bay St at Bloor St W",
        "stop_type": "subway_entrance",
        "h3_r10": "8a2a1072b59ffff",
        "neighbourhood_id": "75",
        "neighbourhood_name": "Yonge-Bay Corridor",
        "scores": {
          // 3 day_types × 24 hours = 72 entries
          "weekday": {
            "21": { "score": 73.2, "rank_pct": 0.41 },
            "22": { "score": 71.8, "rank_pct": 0.39 },
            // ...
            "04": { "score": 52.1, "rank_pct": 0.18 }
          },
          "fri_sat": { /* ... */ },
          "sun": { /* ... */ }
        },
        "factors_22h_weekday": {
          "lighting": { "value": 78.4, "raw": { "lights_75m": 12, "avg_lumens": 5800 } },
          "crime": { "value": 64.1, "raw": { "incidents_36mo_night": 7 } },
          "eyes_on_street": { "value": 81.2, "raw": { "open_venues_150m": 9 } },
          "isolation": { "value": 70.0, "raw": { "poi_count_200m": 24 } },
          "wait_exposure": { "value": 68.0, "raw": { "expected_wait_min": 7.5 } },

```

```

    "sightline": { "value": 60.5, "raw": { "occlusion_ratio": 0.31 } },
    "crossing": { "value": 55.0, "raw": { "ksi_density": 0.04 } },
    "disorder_311": { "value": 72.0, "raw": { "complaints_90d": 3 } },
    "alcohol": { "value": 50.0, "raw": { "outlets_200m": 4 } },
    "streetscape_vision": { "value": 66.0, "raw": { "model_v": "svs-0.3" } }
  },
  "provenance_url": "/data/provenance/11234.json"
}
]
}

```

Provenance files (one per stop) live separately so the main GeoJSON stays under 5 MB gzipped.

5. ML / Scoring Model Specification

5.1 Factor Catalogue

All factors normalized to [0,100] where **100 = safest**. Composite is a weighted sum.

#	Factor	Definition	Data Source(s)	Computation	Weight range
1	Lighting	Density × quality of street lights within 75 m	DS-04	$\log_{10}(\text{count} + 1)$ × avg_lumens_norm → 0–100	0.15
2	Crime History	Decay-weighted night MCI incidents within 250 m, last 36 mo	DS-03	Σ incident × $\exp(-t/12)$, = 12 mo; transform 100 - percentile_rank × 100	0.15–0.25
3	Eyes-on-Street	Count of open venues at query bin within 150 m	DS-10	Parse opening_hours, count active; log-scale	0.10–0.15

#	Factor	Definition	Data Source(s)	Computation	Weight range
4	Isolation (inverse)	Inverse of POI + dwelling density within 200 m	DS-10, DS-09	$100 - \text{density_z_norm} \times 100$	0.05–0.10
5	Wait Exposure	$E[\text{wait min}] \times \text{isolation factor}$	DS-01, DS-02	static GTFS headway/2 with RT override; multiplied by $(100 - \text{isolation})/100$	0.10–0.15
6	Sightline	Building / wall occlusion in 50 m forward arc from stop	DS-10 (OSM buildings)	Ray-cast 8 azimuths, % blocked	0.05
7	Crossing Danger	$\text{KSI density} \times \text{distance to nearest signalized crossing}$	DS-07, DS-10	Composite penalty	0.05
8	311 Disorder	Count of disorder-tagged 311 requests, 90 d, 200 m	DS-06	Categories: graffiti, noise, drug paraphernalia, illegal dumping	0.05
9	Alcohol Outlet Density	AGCO premises within 200 m, capped	DS-11	$\min(\text{count}, 10) / 10$, inverted	0.05

#	Factor	Definition	Data Source(s)	Computation	Weight range
10	Streetscape Vision Score	CV model on Mapillary nighttime imagery (luminance, frontage, occlusion)	DS-12	DINOv2 features + linear head → 0–100	0.05–0.10

5.2 Weight Methodology

Weights are determined by a **three-way blend**:

1. **Theory weight (40%)** — From peer-reviewed urban-safety literature (CPTED, Loukaitou-Sideris 1999, Newton 2014). Yields a prior `w_theory`.
2. **Data weight (40%)** — Logistic regression of factor values against nighttime MCI occurrence (binary label, balanced sample). Coefficients normalized to sum to 1: `w_data`.
3. **Survey weight (20%)** — Stretch goal: a Google Form survey of ~100 women rating factor importance 1–5. Mean importance normalized: `w_survey`. If not collected, this 20% is redistributed pro-rata.

Final: `w_final = 0.4 × w_theory + 0.4 × w_data + 0.2 × w_survey`

Weights file is checked in (`config/weights.yaml`) and the methodology page renders it.

5.3 Temporal Dimension

- **Hour bins:** 21, 22, 23, 0, 1, 2, 3, 4 (8 night bins for the index). Day-time bins (5–20) still computed for parity but de-emphasized in the UI.
- **Day-types:** weekday (Mon–Thu), `fri_sat`, `sun`.
- **Seasonality:** v1 ignores it (winter dark earlier, summer late dusk). Documented as known limitation; v2 candidate.

5.4 Validation Methodology

Method	What it tests	How
Holdout AUROC	Does the composite separate crime-incident vs. non-crime locations?	80/20 spatial split (grid-based, not random), AUROC 0.70 target

Method	What it tests	How
Spearman vs. ward-level safety surveys	Concurrent validity	If Toronto Vital Signs / Toronto Foundation safety perception survey is geocoded, compute
Sensitivity analysis	Are scores robust to weight perturbation?	Monte Carlo: 1000 trials of $\pm 20\%$ weight noise; report 95% CI band of stop ranks
Bias audit	Does the index disadvantage low-income / racialized neighborhoods?	Score vs. income decile residual plot; Theil index of within/between-group variance
Face validity	Do scores match local knowledge?	Manual spot-check of top-5 best and worst stops with team’s lived knowledge of TO

5.5 Model Versioning

- Pipeline emits `model_version` in every output (e.g., `v0.3.1`).
- Versions follow SemVer of *the scoring contract*: MAJOR = factor list change, MINOR = weight change, PATCH = bug fix.
- All prior versions kept in `dist/archive/v{version}/` for reproducibility.

6. Frontend Specification

6.1 Pages / Routes

Path	Component	Purpose
<code>/</code>	<code>MapView</code>	Hero: full Toronto map with time slider
<code>/route</code>	<code>RouteView</code>	Origin/destination + time $\rightarrow$ safety-ranked options + LLM audit
<code>/stop/:id</code>	<code>StopDetail</code>	“Last 200m” microview + factor breakdown + provenance
<code>/methodology</code>	<code>MethodologyPage</code>	Weights, formulas, validation, limitations

Path	Component	Purpose
/equity	EquityPage	Score-vs-income narrative + interactive panel
/simulator	PolicySimulator	Sliders to test interventions
/about	AboutPage	Team, license, contact, EOI text
/download	DownloadsPage	CSV / GeoJSON downloads + API doc

6.2 Component Tree (abbreviated)

```

<App>
  <Layout>
    <Header> (nav, mode toggle)
    <Outlet/>
    <Footer> (attributions)
  <MapView>
    <Mapbox basemap (dark)>
    <DeckGLOverlay>
      <ScatterplotLayer stops />
      <HeatmapLayer hex r10 context />
      <IconLayer subway-stations />
    <TimeSlider/>
    <DayTypeToggle/>
    <EquityModeToggle/>
    <LegendPanel/>
  <StopDetail>
    <MicroviewMap 200m/>
    <FactorBars/>
    <ProvenanceList/>
    <LLMNarrationCard/>
  <RouteView>
    <OriginInput/> <DestInput/> <TimePicker/>
    <RouteSummary/>
    <LLMRouteAuditor streaming/>
  <PolicySimulator>
    <InterventionSliders/>
    <CostEstimator/>
    <EquityDeltaReadout/>
  <VoiceMode floating button/>

```

6.3 Key User Flows

Flow 1 — Quick map lookup (P0) 1. User lands on /. 2. Default state: weekday, 22:00, full map view. 3. User drags time slider → 02:00. 4. Stops recolor within 100 ms. 5. User clicks a red stop. 6. Sidebar opens with factor breakdown + provenance link. 7. User clicks “Why this score?” → modal with raw data deep links.

Flow 2 — Route safety (P1) 1. User clicks “Plan a route” → `/route`. 2. Enters origin (geocode autocomplete), destination, time. 3. Submits. 4. UI shows 1–3 route options ordered by aggregate safety score. 5. LLM narration streams in: “Segment 2: Wellesley between Yonge and Bay...” 6. User taps a segment to drill into a stop.

Flow 3 — Policy simulation (P1) 1. User toggles “Policy mode” → `/simulator`. 2. Selects “Add lighting” tool, paints a neighborhood polygon. 3. UI re-scores affected stops in < 500 ms. 4. Readout: “+47 stops moved out of bottom quartile. Estimated cost: \$1.2M. Equity benefit: 78% accrues to bottom-3 income deciles.” 5. User clicks “Share scenario” → URL with serialized state.

Flow 4 — Voice query (P2) 1. User taps mic button. 2. Speaks: “I’m at Bay station, going to King and Spadina, is it safe?” 3. Browser STT → query parser → route engine. 4. Audio response narrates safety summary + recommends route variant.

6.4 State Management

- **Zustand store** with these slices: `mapState` (viewport, layers visible), `timeState` (hour, day_type), `selectionState` (selected stop, route), `simulatorState` (interventions), `equityMode` (boolean + variant).
- **URL state**: every meaningful selection is reflected in the URL via `nuqs` or hand-rolled hash router so links are shareable.
- **No global Redux**; no Context-API gymnastics.

6.5 deck.gl Layer Specification

Layer	Class	Source	Style
L1	<code>ScatterplotLayer</code>	<code>scores.geojson</code>	<code>radius = 6 px,</code> <code>color =</code> <code>viridis(score),</code> <code>opacity 0.85</code>
L2	<code>HexagonLayer</code> (context)	<code>hex_aggregates.json</code>	<code>extruded false,</code> <code>elevationScale 0,</code> <code>opacity 0.3</code>
L3	<code>PathLayer</code> (route)	route response	<code>width 6, color =</code> <code>score-gradient</code>

Layer	Class	Source	Style
L4	IconLayer (subway markers)	static	iconAtlas /icons/subway.png
L5	PolygonLayer (equity)	neighbourhoods.geojson	fill= residual color, only in equity mode
L6 (detail)	IconLayer (lights, CCTV)	within selected stop buffer	small icons

6.6 Accessibility Requirements (WCAG 2.1 AA)

- All interactive controls keyboard reachable.
- Map has an **alternative tabular view** at `/data/table` for non-visual access.
- Color is **never the only encoding** — every score is also numeric and verbally narratable via LLM.
- Focus indicators with 3:1 contrast.
- All non-text content has text alternative.
- Form labels explicit; errors announced via `aria-live`.
- Touch targets 44×44 px.

7. Development Plan

Total available calendar days: **16** (May 15 → May 30).

7.1 Sprint 1 — Pipeline & Core (May 15 – May 22, 8 days)

Day	Task	Owner	Definition of Done
15	Repo bootstrap; CI; Python + Node toolchain; docs scaffolding	Jasdish	<code>make test</code> green; CI passes; docs/SDLC.md committed
15	Mockup & wireframes; design tokens; basemap style	Tiana	4 wireframes; Mapbox style URL
16	Ingest module: TTC GTFS, MCI, Lighting, OSM, 311	Jasdish	Raw files cached; manifest.json populated

Day	Task	Owner	Definition of Done
16	React app skeleton; routing; deck.gl map renders 12k random points	Tiana	App runs locally; placeholder layer visible
17	Clean + Transform stages; H3 grid; per-stop feature table	Jasdish	gold/stop_features.parquet validates against pandera schema
17	StatCan census ingest + spatial join for equity	Tiana	Income joined to hex grid; CSV exported
18	Factors 1–5 implemented (lighting, crime, eyes, isolation, wait)	Jasdish	All factors return [0,100], unit-tested
18	Time slider, day-type toggle, legend	Tiana	Time slider drives recolor; smooth
19	Composite score + provenance JSON; weights.yaml	Jasdish	scores.geojson generated end-to-end
19	Stop detail panel + factor bars	Tiana	Clicking stop opens panel with real data
20	Factors 6–9 (sightline, crossing, 311, alcohol)	Jasdish	All factors present; tests pass
20	Provenance modal + deep links	Tiana	Each factor row links to dataset URL
21	First public deploy to Vercel preview	Both	URL works on phone; team review
22	Sprint review; cut Streetscape Vision if behind	Both	Decision logged; weights re-calibrated

7.2 Sprint 2 — Differentiation & Polish (May 23 – May 30, 8 days)

Day	Task	Owner	Definition of Done
23	Streetscape Vision factor (Mapillary + DINOv2)	Jasdish	At least 5,000 stops scored; fallback documented for the rest
23	Equity Mode UI + residual computation	Tiana	Toggle works; narrative sidebar present
24	Policy Simulator backend (re-score function)	Jasdish	Affected-stops API < 500ms
24	Policy Simulator UI sliders + readout	Tiana	Live re-score visible
25	LLM Route Auditor (Anthropic API + tool use)	Jasdish	Stream narration constrained to factor data
25	Route view UI + geocoding (Mapbox or Nominatim)	Tiana	End-to-end origin→dest→narration
26	“Last 200m” microview	Tiana	Detail map with lights/CCTV/businesses
26	Validation: AUROC + sensitivity analysis	Jasdish	Numbers in methodology page
27	Accessibility audit + fixes; voice mode (P2 if time)	Tiana	axe-core 0 critical; voice optional
27	Performance pass (bundle, lazy load, gzip)	Jasdish	Lighthouse 90 across the board
28	Technical report writing	Both	6-page report drafted
29	Demo video recording + edit	Both	3-min mp4 in /dist
30	Final QA, fix list burn-down, submit	Both	Submission form complete; tag v1.0.0

7.3 Definition of Done — Universal

Every task is “done” only when: 1. Code merged to `main` via PR. 2. CI green (lint + tests + build). 3. Documentation updated (README, methodology, or ADR as relevant). 4. Visible in the deployed preview URL. 5. Reviewed by the other team member.

7.4 Milestones

Date	Milestone	Gate
May 18	M1 — Pipeline emits first end-to-end scores.geojson	go/no-go on weight methodology
May 22	M2 — Public preview deployed	go/no-go on Sprint 2 stretch features
May 26	M3 — All P0 + P1 features live	feature freeze begins
May 28	M4 — Code freeze + technical report draft	only bug fixes after this
May 30	M5 — Submission complete	tag, demo, submit form complete

8. Testing Strategy

8.1 Unit Tests

Module	Tool	Coverage target	Examples
<code>starsai.ingest</code>	pytest + responses	70%	mock CKAN response → correct parquet rows
<code>starsai.clean</code>	pytest	80%	CRS conversions; dedupe; null handling
<code>starsai.transform</code>	pytest	80%	H3 cell assignment; buffer math
<code>starsai.score.factors*</code>	pytest*	90%	Each factor returns float in [0,100]; edge cases (zero data, all-data)

Module	Tool	Coverage target	Examples
<code>starsai.score.composite</code>	<code>pytest</code>	90%	Weighted sum correctness; weight normalization
Frontend hooks	Vitest + React Testing Library	60%	Time slider state, equity toggle, URL sync
Frontend components	Vitest + RTL	50%	FactorBars renders, ProvenanceList expands

8.2 Integration Tests

- **Pipeline E2E:** nightly `make all` on fixture data → assert output schema + spot values.
- **Frontend E2E (Playwright):** load app, drag slider, click stop, open provenance — assert DOM and screenshots.
- **API E2E:** hit `/api/route-score`, `/api/llm-narrate` with golden inputs, assert JSON schema and latency bounds.

8.3 Data Quality Checks

Check	When	Action on fail
Schema validation (pandera)	every pipeline stage	abort with diff
Row-count delta vs. previous run	post-ingest	warn if > 20% delta, require ack
Null-rate per column	post-clean	abort if any required > 1% null
Geographic bbox sanity	post-transform	abort if any stop outside Toronto bbox
Score distribution drift (KS test)	post-score	warn if KS > 0.1 vs. previous version

8.4 Visual Regression

- Playwright + `pixelmatch` snapshots of 8 key views.
- Run on every PR. Diffs > 1% trigger reviewer attention.

8.5 Performance Benchmarks

Metric	Target	Tool
Pipeline <code>make all</code>	15 min	<code>time make all</code> in CI
Lighthouse Performance	90	Lighthouse CI
Lighthouse Accessibility	95	Lighthouse CI
Map FPS during pan	50	manual via Chrome perf, recorded
LLM narration TTFT	1.2 s	<code>/api/llm-narrate</code> timing

9. Deployment

9.1 CI/CD Pipeline (GitHub Actions)

```
# .github/workflows/ci.yml - on every PR
jobs:
  python:
    runs-on: ubuntu-latest
    steps:
      - uv sync
      - ruff check
      - mypy starsai
      - pytest --cov=starsai --cov-fail-under=70
  web:
    runs-on: ubuntu-latest
    steps:
      - pnpm install --frozen-lockfile
      - pnpm typecheck
      - pnpm lint
      - pnpm test
      - pnpm build
      - pnpm test:e2e
      - lighthouse-ci

# .github/workflows/data.yml - cron 0 6 * * * (nightly)
jobs:
  refresh:
    steps:
      - python -m starsai.ingest --all
      - python -m starsai.clean
      - python -m starsai.transform
      - python -m starsai.score
      - python -m starsai.pack
      - git commit data/dist/* -m "data: nightly $(date +%F)"
      - git push
```

```
# .github/workflows/deploy.yml - on tag v*.*.*
jobs:
  deploy:
    steps:
      - vercel deploy --prod --token=${{ secrets.VERCEL_TOKEN }}
```

9.2 Vercel Configuration

```
// vercel.json
{
  "buildCommand": "pnpm build",
  "outputDirectory": "frontend/dist",
  "functions": {
    "api/llm-narrate.ts": { "maxDuration": 30 },
    "api/route-score.ts": { "maxDuration": 10 },
    "api/gtfs-rt.ts": { "maxDuration": 5 }
  },
  "headers": [
    { "source": "/data/(.*)", "headers": [
      { "key": "Cache-Control", "value": "public, max-age=3600, s-maxage=86400" },
      { "key": "Content-Encoding", "value": "gzip" }
    ]}
  ],
  "crons": [
    { "path": "/api/gtfs-rt-refresh", "schedule": "*/1 * * * *" }
  ]
}
```

9.3 Data Update Workflow

1. Nightly cron in `.github/workflows/data.yml` runs the pipeline.
2. Outputs (`scores.geojson`, `provenance/*.json`) are committed to `data/dist/`.
3. Vercel auto-deploys on the push.
4. CDN caches with `s-maxage=86400` (1 day).
5. Cache busting via filename hash in manifest.

9.4 Rollback Strategy

- All releases tagged (`v1.0.0`, `v1.0.1`, ...).
- Vercel keeps every deployment; one-click promote of previous deployment to production.
- Data outputs archived at `data/dist/archive/v{version}/` so frontend can pin to a known-good dataset version if a regression is found.

10. Risk Register

ID	Risk	Probability	Impact	Mitigation	Owner
R-01	MCI crime data has lat/long offset for privacy (100m); local accuracy degraded	High	Med	Acknowledge in method-ology; use 250m buffer (already > offset); aggregate to hex	Jasdish
R-02	Mapillary nighttime coverage in Toronto is sparse	High	Med	Cap Streetscape Vision factor weight; gracefully degrade to neutral 50 when no imagery; document % stops covered	Jasdish
R-03	Vercel free tier hits function-second limits during demo	Med	High	Pre-compute every static thing; only LLM narration is dynamic; mark LLM as best-effort	Jasdish

ID	Risk	Probability	Impact	Mitigation	Owner
R-04	Anthropic API key cost spike during judging period	Med	Med	Token budget cap per session; cache narrations by route hash; show stale-but-cached on failure	Jasdish
R-05	Scope creep — 11 features pitched, only 8 will ship	High	Med	Strict cut list (Sec 7.2 day 22 review); stretch features explicitly labeled	Both
R-06	One team member sick / unavailable mid-sprint	Med	High	Cross-train: both can deploy and ship hotfixes; daily 15-min standup; commit early commit often	Both

ID	Risk	Probability	Impact	Mitigation	Owner
R-07	Judges interpret index as “redlining” women out of neighborhoods	Med	High	Equity Mode is front-and-center; report has a “How not to misuse this” section; never deliver an actionable individual route avoidance without LLM contextual narration	Tiana
R-08	OSM data quality varies by neighborhood (well-mapped downtown, sparse north)	Med	Med	Use multiple POI sources cross-check; document coverage caveats per ward; show data-density layer in methodology	Jasdish
R-09	TTC GTFS-RT feed downtime during demo	Low	Med	Static GTFS fallback baked in; show “live” badge only when RT recent (< 2 min)	Jasdish

ID	Risk	Probability	Impact	Mitigation	Owner
R-10	Mapbox token quota exceeded under demo load	Low	High	Use Mapbox token + Maplibre fallback to free tiles (CARTO basemap)	Tiana
R-11	Survey for weight-blend doesn't get enough responses	Med	Low	Pre-built fallback: redis-tribute 20% pro-rata across theory + data	Both
R-12	Submission form requires a format we haven't anticipated (e.g., specific file types)	Low	High	Re-read TD2026-Rules on May 20; prepare deliverables in 3 common formats	Tiana
R-13	StatCan 2021 CT income data not fine-grained enough for hex-level equity	Med	Low	Document spatial-mismatch; use areal-weighted interpolation; note as known limit	Jasdish

ID	Risk	Probability	Impact	Mitigation	Owner
R-14	A factor turns out to be perfectly anti-correlated with another (multi-collinearity)	Med	Low	Run VIF analysis; merge or drop redundant factors before final weight blend	Jasdish

11. Competition Submission Checklist

11.1 Required Deliverables

- ☐ **Public GitHub repository** (MIT license, README, install instructions)
- ☐ **Live demo URL** on Vercel production
- ☐ **3-minute demo video** (mp4, 1080p, captions burned in)
- ☐ **Technical report** (PDF, 6 pages, see structure below)
- ☐ **Submission form** filled completely
- ☐ **Data sources & licenses page** (attributions for every dataset used)
- ☐ **Methodology page** (live + in report)
- ☐ **Team page** (Jasdish + Tiana + roles)
- ☐ **Reproducibility statement** (make all works on a clean clone)

11.2 Quality Gates (must pass before submit)

- ☐ Lighthouse Performance 90 on /
- ☐ Lighthouse Accessibility 95 on /
- ☐ All P0 and P1 functional requirements met
- ☐ All 10 factors documented; 8 implemented and live
- ☐ AUROC 0.70 reported in methodology
- ☐ Equity Mode is functional and reviewed
- ☐ No `console.error` in production console on demo flow
- ☐ Demo video re-watched by both team members and approved
- ☐ Tag `v1.0.0` cut, points to deployed commit
- ☐ CHANGELOG.md and VERSION file updated

11.3 Technical Report Outline (6 pages)

1. **Abstract** (½ page) — what, why, results.

2. **Problem & Context** (½ page) — Toronto, women, night transit.
3. **Methodology** (1.5 pages) — factors, weights, temporal model, validation.
4. **System Architecture** (1 page) — compressed version of Section 3 of this SDLC.
5. **Results & Validation** (1 page) — AUROC, sensitivity, equity audit, surprising findings.
6. **Limitations & Ethics** (½ page) — known limits, mis-use guardrails, future work.
7. **References & Attributions** (1 page) — academic + data sources.

11.4 Demo Video Script (3 minutes, 36 lines of voiceover)

Time	Visual	Voiceover
0:00–0:15	Title card	“Toronto Nighttime Transit Safety Index — by STARS AI”
0:15–0:45	Map view, drag time slider 21→04	Hook: “Women restrict night transit more than men. Toronto has the open data to explain why.”
0:45–1:15	Click a red stop, open factor breakdown	“Every score is built from 10 transparent factors. Every number deep-links to the raw open data row.”
1:15–1:45	Equity Mode toggle on	“We don’t just score stops. We expose how income predicts the gap — so the index can’t be used to redline.”
1:45–2:15	Policy Simulator: paint a polygon, add lights	“Drag a neighborhood. Add lighting. See 47 stops move out of the bottom quartile — for \$1.2M.”
2:15–2:45	Route view: type origin/dest, LLM narration streams	“And our route auditor narrates the walk in plain English — backed by the data, not a hallucination.”

Time	Visual	Voiceover
2:45–3:00	Close on URL + team names	“Live now at stars-ai.vercel.app. By Jasdish Singh and Tiana Gaurd. Transit Data Challenge 2026.”

Appendix A — Repository Layout

```

STARSai/
  CLAUDE.md
  README.md
  VERSION
  CHANGELOG.md
  pyproject.toml          # uv
  pnpm-workspace.yaml
  vercel.json
  Makefile
  .github/workflows/
    ci.yml
    data.yml
    deploy.yml
  data/
    raw/                  # downloaded (gitignored, recreated via make)
    bronze/               # ingested parquet
    silver/               # cleaned parquet
    gold/                 # transformed feature tables
    dist/                 # published artifacts (committed)
  pipeline/               # alias of /starsai package
    starsai/
      ingest/
      clean/
      transform/
      score/
        factors/
        composite.py
        provenance.py
      pack/
  model/                  # ML notebooks + DINOv2 head training
  api/                    # Vercel edge functions
    llm-narrate.ts
    route-score.ts
    gtfs-rt.ts

```

```

    dp-report.ts
frontend/
  src/
    pages/
    components/
    hooks/
    stores/
    lib/
  public/
  vite.config.ts
config/
  weights.yaml
tests/
docs/
  SDLC.md           # this file
  METHODOLOGY.md
  ARCHITECTURE.md
  adr/              # Architectural Decision Records

```

Appendix B — Architectural Decision Records (initial list)

- ADR-001: Use H3 r10 over square grid or census tracts
- ADR-002: Vercel over Render (chose: Vercel — better edge + free tier)
- ADR-003: Static JSON + edge functions over FastAPI server
- ADR-004: DuckDB over PostGIS for pipeline (chose: DuckDB — zero-server)
- ADR-005: Deck.gl over Leaflet for 12k+ point rendering
- ADR-006: Anthropic Claude Haiku over GPT-4o-mini for LLM narration
- ADR-007: Three-way blended weights over single-method weighting

End of SDLC v1.0 — STARS AI